

Are Large Language Models Attentive Readers?

Evaluating Multi-Hop Reasoning with Plausible but Incorrect Alternatives

Neeladri Bhuiya¹, Viktor Schlegel², Thomas Winkler³

¹ASUS Intelligent Cloud Services (AICS) Singapore, National University of Singapore

²Imperial Global Singapore

³University of Manchester, United Kingdom

neeladri.bhuiya@u.nus.edu, v.schlegel@imperial.ac.uk, winkler@nus.edu.sg

Abstract

State-of-the-art Large Language Models (LLMs) are accredited with an increasing number of different capabilities, ranging from recommendation over advanced mathematical and reasoning skills to possessing scientific knowledge. In this paper we focus on multi-hop reasoning—the ability to identify and integrate information from multiple textual sources. Given the concerns with the presence of simplifying cues in existing multi-hop reasoning benchmarks, which allow models to circumvent the reasoning requirement, we set out to investigate whether LLMs are prone to exploiting such simplifying cues. We find evidence that they indeed circumvent the requirement to perform multi-hop reasoning, but they do so in more subtle ways than what was reported about their fine-tuned pre-trained language model (PLM) predecessors. We propose a challenging multi-hop reasoning benchmark by generating seemingly plausible multi-hop reasoning chains that ultimately lead to incorrect answers. We evaluate multiple open and proprietary state-of-the-art LLMs and show that their multi-hop reasoning performance is affected, as indicated by up to 45% relative decrease in F1 score when presented with such seemingly plausible alternatives. We also find that—while LLMs tend to ignore misleading lexical cues—misleading reasoning paths indeed present a significant challenge. The code and data are made available at <https://github.com/zawedcvg/Are-Large-Language-Models-Attentive-Readers>.

1 Introduction

Recent developments in the field of language modelling and the introduction of open [?] and proprietary [?] Large Language Models (LLMs) have undeniably advanced the state of the art in Natu-

ral Language Processing (NLP). LLMs have been credited with various understanding and reasoning capabilities, ranging from arithmetic [?], deductive [?] and formal [?, ?] reasoning and possessing general [?], and domain-specific [?] knowledge. Due to their size and generalisation capabilities [?], their evaluation on benchmarks requiring such types of reasoning is typically performed in zero- or few-shot settings on many NLP tasks, without the need for fine-tuning datasets.

These zero- and few-shot capabilities seem to alleviate one of the weaknesses identified with the previous generation of fine-tuning based NLP architectures such as transformer-based [?], and pre-trained language models [?—the reliance on dataset-specific “artefacts” [?, ?] and, as a consequence, lack of generalisation beyond specific datasets. For example, in one of the popular reading comprehension and reasoning benchmarks [?], the majority of questions starting with "How many" can be answered correctly with "2". Following standard finetuning practice and splitting data in train and test randomly, such a simple heuristic will be present in both training and evaluation data, so a fine-tuned model will learn it and obtain high scores, without necessarily performing reasoning. LLMs seemingly circumvent this issue, as they are not finetuned on benchmark data. As such, they are not exposed to simplifying dataset artefacts by design, and it is reasonable to assume that they do not learn to exploit them.

However, while there is a growing body of work investigating the strengths and limitations of LLMs [?], little research has been carried out to validate this assumption, and to investigate whether and to what extent LLMs inherit the "dataset artefact" weaknesses of their fine-tuned predecessors. This is an important research question to pursue, motivated by recent findings on benchmark leakage into pre-training or instruction-tuning data [?], which invalidate the zero-shot setting and potentially al-

low LLMs to learn such dataset artefacts. Another line of research suggests that LLMs tend to "over-reason" [?], perhaps due to "sycophancy" [?], i.e., the tendency to generate the presumably preferred answer over the correct one, leading to complicated reasoning where none is required.

In this paper, we turn our attention to the well-studied capability to perform multi-hop reasoning and reading comprehension— that is, to integrate textual information from multiple different source documents. Typically, this capability is evaluated by asking questions where the necessary information to arrive at the correct answer is spread across multiple documents [?, ?, ?]. It is important to understand to what extent NLP methods possess this capability, as it is required for many real-world tasks, such as retrieval-augmented generation [?] when summarising retrieved documents, and because it is a necessary prerequisite to human-level reading comprehension [?].

Previous work has shown that NLP architectures might possess inadequate capabilities to perform multi-hop reasoning [?]. However, these findings were established before the advent of large language models. To have a clear understanding of the limitations of the capabilities of state-of-the-art research, it is crucial to re-investigate these claims with the current generation of LLM-based approaches [?]. While there is vivid research on (open-book) multi-hop reasoning capabilities of LLMs [?, ?, ?], how well they perform when presented with multiple, seemingly plausible multi-hop reasoning paths remains unclear.

To address this gap, we focus on the capability of LLMs to perform multi-hop reasoning when multiple seemingly plausible answers are present, where only minor details invalidate the alternative. We show that existing methods—calibrated to evaluate pre-LLM architectures—are inadequate to evaluate LLMs, and that LLM reasoning failures are indeed distinct from their fine-tuned PLM predecessors. We present a methodology to generate challenging examples with "plausible distractors" to evaluate LLMs' capabilities to perform multi-hop reasoning when presented with seemingly correct, but ultimately wrong and thus distracting evidence. Our results show that the reasoning capabilities of a range of open and proprietary LLMs, including GPT-4, are affected by these "plausible distractors".

2 Related Work

It has been shown that basic pattern matching [?] and one-hop [?] models can solve a large proportion of questions in multi-hop question answering datasets, presumably because the answer sentence often contains keywords common with the ques-

IMAGE NOT PROVIDED

Figure 1: Our proposed method evaluates the multi-hop reasoning capabilities of Large Language Models by adding seemingly plausible, yet ultimately wrong alternate reasoning paths, impacting the reasoning performance of state-of-the-art LLMs such as GPT-4.

tion, thus negating the need to follow a reasoning path and attend to multiple documents. Particularly HotpotQA [?], due to its multi-hop question design, was the subject of multiple studies. Approaches architecturally incapable of multi-hop reasoning still achieved close to state-of-the-art performance [?, ?], suggesting questions answerable in such a way do not necessitate multi-hop reasoning.

In light of these results, several adversarial attacks have been proposed to check whether the dataset evaluates multi-hop reasoning without exhibiting "shortcuts", by ensuring that the correct answer can only be procured if the evaluated model can retrieve and combine information from distinct reasoning hops. Jiang and Bansal (2019) elicited distracting paragraphs by using the titles of the gold paragraphs and the answer, which are subjected to phrase-level perturbations and word replacement, thus creating a distracting paragraph. Others decomposed the multi-hop questions in multiple single questions [?, ?, ?] (e.g. DecompRC in Figure 2) showed that the—typically BERT- or other PLM-based—fine-tuned SOTA models struggled to answer both sub-questions correctly when answering the complete question, or were distracted by their alterations, suggesting the presence of reasoning shortcuts [?].

By design, these methods bear only negative predictive power [?]: failing to see a performance drop does not imply that the model performs the evaluated capability well, but rather that the methodology might have limited suitability to evaluate the investigated phenomenon, i.e., multi-hop reasoning. As the methodologies presented above focus on fine-tuned models, they assume that multi-hop reasoning is circumvented through simple, lexical similarity-based methods like word matching. For example, Jiang and Bansal (2019) do not consider that their generated paragraphs are isolated, as they contain no explicit reference to other paragraphs in the context, such as a shared named entity. Meanwhile, Ding et al. (2021) only add a single distracting sentence. Thus, simple word matching, which ensures that the final answer is of the same entity type as in the question, can often lead to the correct answer. This might not be sufficient for LLMs, as they—due to their size and emergent capabilities—might circumvent multi-hop reason-

ing by exploiting more subtle textual cues. Indeed, in our empirical study, we show that existing methods, due to these limitations, do not adequately test an LLM’s reasoning capabilities.

Therefore, to analyse an LLM’s ability to reason more adequately, we go beyond the state of the art and introduce a novel method to more effectively evaluate the multi-hop reasoning capabilities of LLMs. Specifically, we ensure the continuity of seemingly plausible alternative reasoning paths, which lead to answers that are ultimately wrong. To succeed, the model is required to pay close attention to small yet important details in the questions and paragraphs. This ability is important practically, for example when an LLM is prompted to evaluate/summarise the outcome of a debate, where both sides will present plausible arguments with only one being ultimately correct [?, ?]. With LLMs increasingly used to judge and improve (other) LLMs’ potentially similar outputs on the same topic [?], it is important to establish, if they possess the necessary prerequisites to do so. More broadly, similar to other works in this line of research, we look at linguistic competence rather than performance [?]: if we accredit multi-hop reasoning capabilities to LLMs, then, similar to humans, we expect them to exhibit these capacities not only in the majority of cases but in edge case scenarios as well, such as when presented with seemingly plausible alternate reasoning paths.

3 Methodology

In this section, we describe our approach to evaluating the multi-hop reasoning capabilities of LLMs. We do so by creating "distractor" paragraphs that present seemingly plausible yet incorrect alternative paths in the reasoning chain while ensuring that this process doesn’t affect the final solution. First, the question is treated as a two-hop question and converted into two sub-questions. This is done to be able to branch out alternative reasoning paths from each of the sub-questions. The sub-questions are analyzed to identify modifiable portions, which are then manipulated to create "distractor" sub-questions that lead to a different answer and thus a different reasoning chain, which is ultimately wrong, as the models are presented with the original, unmodified question. The "distractor sub-questions" are finally used to generate "distractor paragraphs" containing "distractor answers" utilizing an LLM.

The method comprises three main steps: I. Acquiring the main entity, II. Extracting its modifiable details, and III. Creating the distractor paragraphs.

Question: What year did Guns N Roses perform a promo for a movie starring Arnold Schwarzenegger as a former New York Detective?

Sub question 1: Which movie stars Arnold Schwarzenegger as a former New York Police detective?

Sub question 2: What year did Guns N Roses perform a promo for End of Days (answer of the previous question)?

Figure 2: Example of a decomposed multi-hop question.

3.1 Step I: Acquiring the main entity

We use the human-annotated sub-questions from Tang et al. (2021), as exemplified in Figure 2. We define main entities as those that are the focus of the question. For example, in Figure 2, the main entities for the sub-questions would be "movie stars" and "year" respectively. We choose the "main entity" in each sub-question, using a few dependency parse-based rules. Intuitively, we exploit the relations between the "wh"- word and other noun phrases to extract the main entity. Specifically:

- (i) If the "wh" question word WH is the root, and there exists a word A with a dependency $nsubj$ or $nsubj:pass$ with WH as the head, A is the main entity.
- (ii) Alternatively, if there exists a word A with a dependency of type det , $nsubj$, or $nsubj:pass$ with a wh-word WH (a) If A is a noun, A is the main entity. (b) Otherwise, if A is a verb, the word B having a relation $acl:relcl$ with B being the head, we mark B as the main entity.
- (iii) Else, if any word A has a dependency with a word B of type $nsubj$ or $nsubj:pass$, where B is the word with a direct dependency with the wh-word, A is assigned as the main object.

3.2 Step II: Extracting the details

Next, we extract the details that need to be manipulated to create the distractor question. The main idea is to obtain modifiers of any entity in the question other than the main entity (from the previous step). Specifically:

- (i) For any dependency between two words C and D , we check if the dependency is of the form obl , obj , $nsubj$, or $nsubj:pass$. We also ensure that D isn’t the main entity identified in the previous step.
- (ii) If the above rule is satisfied, we check if C or D has a dependency $appos$ with any named entity.

- (iii) If there is no such relation, modifiers of D of the form nummod, amod, nmod, compound, or flat are used to get modifiable parts if the modifier isn't the main entity identified in the previous steps.

We extract the modifiers and not the object they modify for two reasons: First, changing the object often causes the overall question to become nonsensical. Secondly, changing the modifier ensures a minimal yet semantically meaningful modification of the question [?].

3.3 Step III: Creating the distractor paragraphs

After obtaining modifiable parts, we distinguish whether these are Named Entities or not. For each of the named entities, we obtain their type using Qi et al. (2020)'s Named Entity Recognition (NER) processor. We then generate a fake entity of the same type with the help of GPT-4.

Next, for the non-named entities, we use RoBERTa's [?] masked token prediction objective to obtain alternative words. Specifically, we mask the modifiable parts and sample the top ten probable tokens from the language model. To ensure that the new word is sufficiently different yet still plausible given the context, we establish the following constraints empirically:

- Sentence Similarity of the new sequence in comparison to the initial question, as given by the cosine similarity of all-mpnet-base-v2 [?] is < 0.991
- Word similarity under RoBERTa of the original word and the word replacing it is < 0.4
- Perplexity, i.e. the RoBERTa predicted probability of the new sentence, is > 0.001

The new words and named entities are used to create new fake questions. We use these fake questions to create fake question tuples, i.e., fake questions for the different hops. While generating the fake question tuples, we mask the tokens in the second sub-question corresponding to the first sub-question's answer. Next, we feed these fake tuples into GPT-4 and ask it to generate the distractor paragraphs. We generate a pair of distractor paragraphs for each tuple. Figure 3 shows the instantiation of our proposed method on a single example, with the generated distractor paragraphs and the corresponding gold paragraphs. In the attack each of these distractor paragraphs replaces one of the nongold paragraphs, to prevent adding extra tokens and to ensure that the ratio of 2 gold paragraphs and 8 distractor paragraphs of the distractor setting of HotpotQA is maintained.

IMAGE NOT PROVIDED

Figure 3: Instantiation of our proposed method. With "arena" as main entity of sub-question 1, we extract "home" to be replaced with "play-off". Then, we use the modified sequence with the original sub-question 2 (masking the answer "Androscoggin Bank Colisée") as prompt to GPT-4 to generate the distractor paragraphs 1 and 2. The distractor paragraphs generated have "Maple Leaf Arena" as the bridging entity in the false reasoning chain which leads to the wrong answer "4500 spectators".

3.4 Data Quality

Following this procedure, we generate 132 instances of the "other" type, while 547 are created from named entities. To ensure that the generated distractor paragraphs are valid, do not contradict the gold paragraphs, and do not cause contradictions with the label, we randomly sample and inspect 100 named entity-based and all 132 of the "other" examples. For the former, none of the sampled examples were contradictory. For the latter, 13 were found to have either one or both of the distractor paragraphs contradictory—those examples were discarded. Furthermore, we conducted a user study (see Appendix F), which showed that humans have no difficulty extracting the correct answer when given a combination of real and distractor paragraphs. It was also reported that the distractor paragraphs seldom contain contradicting information. We further compare the word count of the adversarial and the original paragraphs to check if the adversarial paragraphs artificially increase complexity through a larger word count. On average, the adversarial paragraphs had a word count of 81.2, slightly lower than the average word count of the original paragraphs, which is 95.95.

Through manual verification, a user study, and the comparison of the word count of plausible paragraphs and their counterpart real paragraphs, we can conclude with high certainty that the plausible paragraphs don't contain contradictory information, and that the drop in performance of the models is due to their inherent weakness and not some artificially added complexity.

4 Experiment Setup

First, we investigate LLM's capabilities and limitations compared to previous PLM-based state of the art. Then, we evaluate the multi-hop reasoning capabilities of LLMs using our proposed methodology. Finally, we conduct an in-depth analysis of what makes reasoning hard for LLMs on our benchmark

and conclude by evaluating state-of-the-art LLMs and prompting techniques. Unless mentioned otherwise, we use the chat models for Llama-2.

Do LLMs suffer from the same flaws as finetuned models? Llama-2-13B [?] is used as the baseline LLM. We evaluate using few-shot prompts, as these allow the model to stick to the expected output format better than zero-shot. This setting is used throughout the paper unless mentioned otherwise. Two styles of prompts were used, normal and chain of thought, as per the strategies discussed in Wei et al. (2023). All reported metrics are measured at token level and averaged across all the instances, following standard evaluation practice [?].

We test the LLMs’ performance when attacked with AddDoc [?], an adversarial attack on HotpotQA for BERT-based models. This is intended to check an LLM’s ability to handle "distracting" paragraphs. SubQA was used to determine if the models could answer the individual questions before answering the entire question. It is a sample of 1000 questions and their sub-questions from the dev set of HotpotQA, with the sub-questions being human-verified. This allows us to evaluate model consistency in answering both the multi-hop question as well as the individual sub-questions correctly. It also allows us to investigate the opposite: When the (more complex) composite question is answered correctly, but either of the (simpler) decomposed questions is answered wrongly, the model might rely on some reasoning shortcuts, discarding sub-question information. Finally, we evaluate if LLMs can retrieve the correct answer when necessary information from one of the gold paragraphs is missing, using the DiRe test set [?].

Do LLMs get distracted by seemingly plausible alternate reasoning paths? As described in Section 3, the attack aims to create paragraphs that provide irrelevant information that is closely related to the property/entity being questioned about. Here, we evaluate a representative sample of open-source and proprietary LLMs, specifically, Llama-2-13B, Llama-2-70B, Mixtral-8x7B-Instruct-v0.1, GPT-3.5 and 4. To contextualise the performance of LLMs to their fine-tuned PLM counterparts, we also finetune a longformer model on the HotpotQA training set and evaluate it on our proposed benchmark (see Appendix for details). Based on the chatbot leaderboard [?] at the time of writing, the best state-of-the-art model was GPT-4. Thus we evaluate GPT-4 to investigate how our findings generalise to stronger models.

What are the effects of the different parameters? Experiments are conducted to check the impact of the method’s parameters on the performance of LLMs. Specifically, the different parameters we investigate are: 1) number of "distractor" paragraphs generated, i.e., two or four;

Table 1: Comparing normal and chain-of-thought prompts using Llama-2-13B as baseline.

Type	F1 score	Precision	Recall	Ans # words (avg)
Normal	0.5077	0.5180	0.575	4.6
CoT	0.4791	0.4682	0.599	5.08

2) whether the distractor paragraphs are generated from the two sub-questions belonging to the same multi-hop question or if the sub-questions belong to two independent multi-hop questions; 3) The type of modifiable portion that is changed in the sub-question, i.e., Named Entity or not; 4) whether the paragraphs, if not generated from two distinct sub-questions, are both generated from the second sub-question.

5 Experiment Results

In this section, we present the results of our experiments, compare them against prior work, and discuss deeper insights. Unless otherwise stated, all reported results of the adversarial attack are statistically significant at $p < 0.05$, determined by conducting a one-sided Student’s t-test.

5.1 Do LLMs suffer from the same flaws as fine-tuned models?

5.1.1 I. Setting up the baseline

Llama-2-13B chat model is used as the baseline for the performance of an LLM in a zero/few-shot setting; results are shown in Table 1. The F1 score indicates that the few-shot setting without chain-of-thought prompting performs best. This is because in the chain of thought setting the model often gives a lengthy explanation, thus reducing precision and F1 score.

5.1.2 II. Reasoning shortcuts using SubQA

Table 2 shows the result of running few-shot Llama-2-13B in the controlled setting on the SubQA dataset. Llama-2 performs much better on the individual sub-questions than the question requiring multi-hops. This finding, in line with analyses focusing on fine-tuned models [?], suggests some inconsistencies in its reasoning capabilities and difficulty in combining information from multiple sources.

Table 3 indicates the performance statistics for individual samples. $F1 > 0.5$ is used here to evaluate a question as correct. The first row consists of questions where the individual sub-questions and the whole question were answered correctly. The second row indicates the questions where the final

Table 2: Results of Llama-2-13B on SubQA dataset

Type	F1 score	Precision	Recall
Original	0.427	0.427	0.507
Sub question 1	0.743	0.761	0.789
Sub question 2	0.693	0.691	0.782

Table 3: Performance statistics on SubQA

Type	Accuracy
All correct	0.414
Correct but sub-questions wrong	0.107
Wrong but both sub-questions correct	0.25

answer was correct despite getting the individual hops wrong, while the third is where the final answer was incorrect despite the individual hops being correct. 10% of the questions were answered correctly without getting both sub-questions correct. This accounts for over 20% of the questions that the model got correct, which is considered model failure by Tang et al. (2021), thus indicating that the model indeed follows some form of shortcuts in its multi-hop reasoning process. However, this percentage is much lower than for PLM-based fine-tuned models, which reach close to 50% [?]. For 25% of the questions, the model got both sub-questions correct but was unable to combine them to give the final answer, thus demonstrating difficulties in bridging and integrating separate information during multiple reasoning hops.

5.1.3 III. Reasoning shortcuts in DiRe

DiRe consists of removing the bridging gold paragraph from the context, with the claim that a model should not be able to answer them under these conditions, and if they are, the examples exhibit a reasoning shortcut exploited by the model. Table 4 shows the results of Llama-2-13B on this. Surprisingly, the model still maintains a decent performance level, confirming that HotpotQA indeed contains several reasoning shortcuts. Seemingly, LLMs- similar to their fine-tuned predecessors- readily exploit such shortcuts despite not being explicitly trained on HotpotQA.

5.1.4 IV. Reasoning failures when presented with distracting paragraphs from AddDoc

Table 5 shows the performance of Llama-2-13B, Llama-2-70B and Mixtral-8x7B-Instruct-v0.1, in few-shot prompt setting, when attacked with the first 2000 examples of AddDoc [?], the most successful method to show reasoning weaknesses of models fine-tuned on HotpotQA, by adding crafted

Table 4: Llama-2-13b performance on DiRe when using a normal (non-CoT) prompt and priming with fewshot examples.

Dataset	F1 score
Original dataset of 4174 examples	68.7
DiRe probe consisting of 5000 examples	44.2

Table 5: F1 score of Llama-2-13b, Llama-2-70b and Mixtral-8x7b-Instruct-v0.1 when attacked with 2000 examples of AddDoc in the few-shot setting.

Model	Original	AddDoc
Llama-2-13B	50.3	51.7
Mixtral-8x7B-Instruct-v0.1	58.0	58.0
Llama-2-70B	53.9	54.6

paragraphs which are lexically similar to the question. Apparently, and in stark contrast to fine-tuned models, LLMs performance does not drop on the benchmark, even slightly increasing for some of the evaluated models. This finding suggests that the reasoning shortcuts exploited by LLMs are indeed less obvious than simple lexical overlap, thus further motivating the need for a more sophisticated method to evaluate multi-hop reasoning, such as those proposed in this paper.

5.2 Do LLMs get distracted when faced with seemingly plausible alternatives?

Table 6 shows the results of various open- and closed- source LLMs using our proposed benchmarking method. All models show a significant drop in their F1 scores and their Exact-Match (EM) scores. Importantly, this seems to be a model property rather than an artefact of the prompting technique, as the behaviour persists across different prompting methods (see Appendix H). Furthermore, even GPT-4 exhibits a drop of 14 points in F1 under the strongest adversarial attack setting i.e., when adding four adversarial paragraphs (see Appendix G). This is remarkable, as the benchmark was partially generated with GPT-4 in the loop. This highlights the feasibility of our method to evaluate a model using an equally strong model as an adversary, a property that other benchmarks tend to lack [?, ?].

5.3 Analysing the effects of different parameters

Next, we investigate which settings contribute most to the drop in performance.

Table 6: Results of Llama-2-13B, Mixtral-8x7B-Instruct-v0.1, Llama-2-70B, GPT-3.5 and longformer (finetuned on the training set) on the original HotpotQA dev set (ori) and our adversarially constructed examples (adv). All the tests for the LLMs are done in the few-shot chain of prompt setting. EM and F1 Performance Scores are reported. F1 scores are further broken down by (left to right): the number of "fake" paragraphs; whether "fake" paragraphs are related; the type of entity modified, if adversarial paragraphs are unrelated, and if both the adversarial paragraphs are generated from the second sub-question of two different fake sub-question pair.

	EM	Overall		Paragraph Count		Paragraph Related		Modified	Type	Second	Sub-Q only
		ori	adv	2	4	Yes	No	Named	Other	No	Yes
Llama-2-13B	30.9	45.8	23.6	47.6	40.9	47.3	43.6	41.7	46.6	45.9	48.3
			-7.2	33.8	-12.0	36.5	-11.1	26.1	-14.7	32.1	-15.2
36.6	-7.0	22.2	-19.5	35.8	-10.0	33.8	-12.0	40.6	-7.7		
Mixtral-8x7B-Instruct-v0.1	50.4	68.1	34.8	67.9	68.7	67.6	69.0	60.4	69.5	68.2	69.5
			-15.6	48.4	-19.7	50.1	-17.7	43.4	-25.3	46.2	-21.1
52.0	-17.0	42.3	-18.1	49.5	-20.0	48.4	-19.7	53.9	-15.6		
Llama-2-70B	54.1	67.7	40.4	66.1	72.7	65.3	71.6	51.3	70.5	67.7	69.8
			-13.6	53.2	-14.5	53.9	-12.1	51.3	-21.4	49.7	-15.6
58.8	-12.8	40.5	-10.8	55.4	-15.1	53.2	-14.5	59.7	-10.1		
GPT-3.5	63.4	77.2	39.9	76.6	78.5	75.3	80.1	72.9	77.8	77.2	80.5
			-23.4	52.7	-24.4	56.0	-20.6	43.1	-35.4	51.3	-23.8
54.9	-25.2	49.7	-23.3	53.2	-24.6	52.7	-24.4	64.3	-16.1		
longformer	71.5	82.1	51.9	81.0	85.1	80.2	84.6	75.9	83.1	82.1	84.7
			-19.5	62.2	-19.8	66.1	-14.8	50.7	-34.4	62.6	-17.6
61.6	-23.3	62.9	-13.0	62.1	-21.0	62.2	-19.9	61.0	-23.7		

5.3.1 Count of distractor paragraphs

As we can modify the number of alternate reasoning chains, and thus generate distractor paragraphs, it is worthwhile investigating whether increasing their number leads to decreased performance. Table 6, "Paragraph count" columns, shows the results of the various models in the chain of thought few-shot setting when facing two or four distractor paragraphs, respectively. Indeed, the higher the number of adversarial paragraphs, the more the model struggles, with an additional decrease of about 10 F1 points for every fake reasoning chain on average.

5.3.2 Are the paragraphs related?

As our method creates fake sub-questions that are used to generate distractor paragraphs, we can modify if the paragraphs to be used in the attack belong to the same fake question pair or not. If not, the attack will use paragraphs from different pairs but will ensure that if k adversarial paragraphs are being added, $k/2$ are generated from the first sub-question and the other from the second sub-question. This is useful to check if models struggle because of the presence of alternate multi-hop reasoning chains, or if the difference in performance is attributed to distractor paragraphs containing similar but otherwise unrelated information.

Table 6, columns "Paragraph Related" shows the performance of the models in this setting. For Llama-2-13B, Mixtral-8x7B-Instruct-v0.1, and Llama-2-70b, related paragraphs, and therefore complete alternate reasoning chains, cause a larger drop than unrelated distractor paragraphs. Interestingly, GPT-3.5 exhibits the opposite behaviour, performing slightly worse when an alternate rea-

soning chain does not connect the distractor paragraphs.

5.3.3 Modified type

Because the main entity of the question can be either part of a Named Entity or not, we can distinguish model performance between these settings. Table 6, columns "Modified Type", shows the results of this test. Aside from Llama-2-13B, which performs significantly worse on Named Entities, the differences are not statistically significant, indicating that both distractor types seem to be equally difficult.

5.3.4 Are the paragraphs unrelated and only belong to the 2nd subquestion?

We have shown that (with the exception of GPT-3.5) examples containing fake paragraphs related by a seemingly alternate reasoning chain are harder for LLMs to process correctly. Similarly, we can investigate if fake paragraphs that are generated purely from the second sub-question add further complexity. Since the paragraph generated from the second sub-question is the only paragraph that contains an entity of the same type as the actual answer, the rationale is to investigate what contributes more to hard multi-hop reasoning: producing seemingly alternate reasoning chains or just adding adversarial paragraphs similar to the paragraph answering the second subquestion. We ensure that the number of adversarial paragraphs, generated using our method, is the same in both settings.

As can be seen in the last column of Table 6, "Second Sub-Q only", all LLMs perform worse

when the paragraphs are not generated from the second sub-question only, thus adding further evidence to the hypothesis that examples with seemingly plausible alternate reasoning chains are indeed harder for LLMs to process correctly. Additionally, only the fine-tuned longformer model exhibits the opposite behaviour, suggesting that PLM-based fine-tuned models indeed tend to learn more simple word-matching type heuristics, as generating multiple paragraphs from the second sub-question results in more fake paragraphs that are lexically similar to the question and answer sentence. This adds further evidence that there is a need to reevaluate the weaknesses of LLMs, as insights derived from PLMs do not necessarily carry over.

The second sub-question-only setting is most similar to AddDoc [?] and other existing attacks on HotpotQA. However, unlike for AddDoc, all LLMs still show a drop in performance. This demonstrates the effectiveness of generating adversarial paragraphs by changing minute details extracted from the question, surpassing the impact of existing attacks. The paragraphs generated in this manner challenge the LLMs more effectively, highlighting their susceptibility to being "blinded by nuance".

6 Conclusion

We explored whether LLMs can perform multi-hop reasoning when presented with seemingly plausible yet ultimately incorrect reasoning paths. To do so, we conducted an extensive evaluation to show how LLMs' multi-hop reasoning abilities differ from the previous generation of PLM-based NLP methods relying on fine-tuning. We found that existing adversarial attacks are inadequate to probe the capabilities of LLMs; thus we introduced a simple yet powerful framework based on generating paragraphs that contain seemingly plausible yet wrong alternative reasoning chains, compatible with any benchmark that requires multi-hop reasoning. Our extensive empirical study shows that all evaluated LLMs (including GPT-4) struggle to succeed on the proposed benchmark. The framework facilitates the generation of adversarial paragraphs, enabling the creation of more rigorous tests which could lead to more robust models. Datasets augmented with such adversarial paragraphs could allow the models to move away from learning nonrobust features like basic lexical matching and enable improved reasoning capabilities. We release data and code to the wider research community on Github: [https://github.com/zawedcvga/](https://github.com/zawedcvga/AreLarge-Language-Models-Attentive-Readers) **AreLarge-Language-Models-Attentive-Readers**.

7 Limitations

The main limitation of the proposed method is that it requires the question to be broken down into its sub-questions. Specifically, we use Tang et al. (2021)'s SubQA dataset, but existing question decomposition techniques like Min et al. (2019b) and Perez et al. (2020) can be used to adapt the framework to all HotpotQA questions or any other dataset that deals with multi-hop reasoning. Furthermore, we use the same algorithm for all types of questions to generate seemingly plausible alternate reasoning paths. However, datasets such as HotpotQA distinguish between different types of multi-hop reasoning, e.g. bridge and comparison. Relying on this knowledge, more sophisticated methods to create seemingly plausible alternate reasoning paths could be developed. Although we perform extensive tests to ensure that the current method generates adversarial paragraphs that do not contradict the gold paragraphs, there is no formal guarantee for it.

A System Prompt for Q/A task

You are a helpful, respectful, and honest question-answering assistant. You will be given a context and a question. Answer the question using only the context. You will break the questions into sub-questions. You will then use these sub-questions to get to the final answer. The final answer must have 'Final Answer: ' prepended to it. Thus your output will be in the following format: Sub-question 1: [subquestion 1] Answer: [answer 1] Sub-question 2: [subquestion 2] Answer: [answer 2] Sub-question n: [subquestion n] Answer: [answer n] Final Answer: [final answer] The final answer should be limited to 5 words with just the answer and no explanation/information. Here are some past conversations: Context: Question: 'Which government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tell?' Sub-question 1: Which woman portrayed Corliss Archer in the film Kiss and Tell? Answer: Shirley Temple. Sub-question-2: Which government position was held by Shirley Temple? Answer: Chief of Protocol. Final Answer: Chief of Protocol. Context: Question: What is the name of the fight song of the university whose main campus is in Lawrence, Kansas and whose branch campuses are in the Kansas City metropolitan area? Sub-question 1: Which university has its main campus in Lawrence, Kansas and has branch campuses in Kansas City metropolitan area? Answer: University of Kansas Sub-question 2: What is the name of the fight song of University of Kansas? Answer: Kansas Song Final Answer:

B System Prompt for creating fake paragraphs

You are a helpful and respectful fake paragraph generating assistant. You will be given two questions, a few supporting paragraphs, and two words you need to avoid. You will first give a fake answer for the first question. The fake answer should not be the same as any of the two words that need to be avoided. Generate a fake paragraph using the information from the first question and the fake answer generated. The answer and information should not be related to any real-life entity. The paragraphs generated must match the tone of the given two paragraphs. Furthermore, the two paragraphs generated must not contradict any of the information in the supporting paragraphs provided by the user. Use the fake answer generated for the first question to replace all instances of '[answer]' in the second question. Use the newly generated question and generate a fake answer for it. Ensure that the fake answer generated is not the same as any of the provided words you need to avoid. Similar to the first question, use the fake answer and the question to generate a fake paragraph. You will generate the fake paragraphs as if they were part of a Wikipedia article. You must maintain a neutral and informative tone. Generate the two paragraphs as separate articles about 75-100 words each. All the answers and paragraphs must be made up of fake names and fake information. The information/names should not reference anyone in real life. Generate exactly one paragraph for each question. Remember to replace all instances of '[answer]' with the answer from the first question and adjust the paragraphs accordingly. However, you must not mention the fact that the details/entities in the paragraphs are fake/imaginary.

C System prompt for creating fake named entities through GPT-4

You are a helpful, respectful and honest fake named entity generator. You will be given upto 20 different entity types along with an example of that type. For each of the entity types, generate another named entity different of the same entity type given the named entity. There are a total of 18 different entity types. The different types and their definitions are as given below:

PERSON: People, including fictional

NORP: Nationalities or religious or political groups

FACILITY: Buildings, airports, highways, bridges, etc.

ORGANIZATION: Companies, agencies, institutions, etc.

GPE: Countries, cities, states

LOCATION: Non-GPE locations, mountain ranges, bodies of water

PRODUCT: Vehicles, weapons, foods, etc. (Not services)

EVENT: Named hurricanes, battles, wars, sports events, etc.

WORK OF ART: Titles of books, songs, etc.

LAW: Named documents made into laws

LANGUAGE: Any named language

DATE: Absolute or relative dates or periods

TIME: Times smaller than a day

PERCENT: Percentage (including

"MONEY: Monetary values, including unit

QUANTITY: Measurements, as of weight or distance

ORDINAL: "first", "second"

CARDINAL: Numerals that do not fall under another type

For each of the provided examples, you will generate one named entity of the same type.

Ensure that your final count of entities is equal to the number of entities in the given prompt. Use indices to help with keeping the count.

D Dependency type definitions

Table 7 consists of definitions of the dependency relations used in the attack. All the definitions are based on De Marneffe et al. (2014).

E Reproducibility

The parameters used for fine-tuning the longformer model are

Batch size: we use batch size of 64 for the longformer model. Learning Rate: We set learning rate to $3e^{-5}$, as it was reported to work best for the transformer training. Train Epochs: We train on HotpotQA for 3 training epochs. Maximal answer length: we set `max_answer_length = 30` when obtaining predictions.

All experiments were carried out on a single NVIDIA Titan RTX GPU

F User study to verify adversarial paragraphs

To verify that examples don't influence gold labels, a user study was conducted involving 5 participants, all of whom had at least college level education. Each participant received the same random sample of 49 questions from the adversarial

dataset. It was ensured that the 49 questions were not from the 100 samples that we manually verified. For each question, participants were provided with two sources of information:

Relevant lines from the gold paragraph. Relevant lines from adversarial paragraphs intended to distract from the correct answer.

The selection of relevant lines followed specific criteria. For the gold paragraphs, we utilized the line numbers identified by the HotpotQA dataset as containing relevant information for the answer. For the adversarial paragraphs, we employed a different approach. During the generation of these paragraphs by GPT-4, plausible answers to sub-questions were crafted. Only sentences containing one of these answers were included, as these would be the sentences that provided information with the potential to mislead the model.

Each question had 4-5 options. One was the correct answer, two were the answers to the sub-questions for the plausible paragraphs, and the rest were titles of the supporting facts" from HotpotQA if these were not already included in the options. After each question, the user was asked if the two sources of information contained contradicting information. The user was given the following prompt

Welcome to our user study! In this study, you will be asked to answer 50 questions. Each question will be accompanied by two sources of information. Please read both the question and the provided sources carefully before selecting your answer. If you encounter any contradictions between the two sources that make it impossible to answer the question accurately, please select "Yes" for "Was there any contradicting information?" Otherwise, select "No". Here is an example of contradictory information: - Source A: "The Kellock-Taschereau Commission was appointed by Adrian Holloway." - Source B: "The Kellock-Taschereau Commission was appointed by Lyon Mackenzie." Note: Do not put "Yes" for contradicting information if there is lack of information. You can make assumptions about who the pronouns refer to if the prior isn't mentioned. Thank you for your participation and careful attention!

Table 8 shows three different metrics:

Average Accuracy: The percentage of questions answered correctly by the participants. Accuracy-Combined: A question is given 1 point if more than 3 participants answered it correctly, and 0.5 points if exactly 2 participants answered correctly and no incorrect answer got more than 2 votes. Accuracy-UB: Adapted from the HotpotQA paper, this metric checks if any of the users were able to answer a particular question correctly.

If a user marks a particular question as containing a contradiction, their answer is marked as incorrect. We use Accuracy-Combined rather than the

Average Accuracy as it better deals with the variance in user answers due to misreading certain information. Accuracy-Combined and Accuracy-UB closely follow the results of human-evaluation on HotpotQA [?]. While it is true that our test is simpler than HotpotQA, as the relevant lines are already provided, this test provides strong evidence to suggest that humans aren't affected by the adversarial paragraphs.

Table 9 shows the number of questions marked as contradictory with a confidence level of greater than 40% . The following questions were marked as contradictory with a confidence > 40% of being contradictory: link. We have gone through the 5 questions marked as contradictory, and just one of them was found to have contradicting information. We attribute the marking of these questions as contradictory to human error and waning attention, which is often present in crowd-sourcing experiments. As per informal user feedback, finding the correct answer was "a bit tricky" at times. Crucially, despite being tricky, they weren't reported to have contradicting information. Through the accuracy metrics and the count of questions marked as contradictory under different confidence levels, we can conclude with certainty that these distractors do not affect the gold labels and are not an issue for humans.

G Performance of SOTA LLM

To see how well our method generalises to better models, we evaluate GPT-4, the best-performing LLM at the time of writing. GPT-4 was tested on 250 examples (due to cost constraints), where fake paragraphs are related by alternate reasoning chains, using Named Entity as the main entity type and alternating between two and four fake paragraphs. Table 10 shows that GPT-4 is more resilient to the attack as compared to the other LLMs that were tested. However, it still exhibits a drop of 14 points in F1 under the strongest adversarial attack setting i.e., related paragraphs, four adversarial paragraphs.

H Do existing techniques make models more robust?

Table 11 shows the results of running more advanced prompting methods than naive chain-of-thought reasoning, such as instructed chain-of-thought prompting [?] and self-consistency [?] on the Llama-2-13B and Llama-2-70B. The setting is 2 plausible paragraphs with the modifiable portion as "other". While self-consistency leads to a smaller decrease in F1 score under the attack, the gains

in robustness (4.2 F1 points) are limited. Instruct prompting on the other hand doesn't provide any relevant improvements. This suggests that our findings unveil a behaviour of LLMs that cannot be corrected simply by using more advanced prompting techniques.

Table 7: Definitions based on Universal Dependencies

Term	Definition
nsubj	nominal subject (nsubj) is a nominal which is the syntactic subject and the proto-agent of a clause.
nsubj:pass	A passive nominal subject is a noun phrase which is the syntactic subject of a passive clause.
obl	The obl relation is used for a nominal (noun, pronoun, noun phrase) functioning as a non-core (oblique) argument or adjunct.
obj	The direct object of a VP is the noun phrase which is the (accusative) object of the verb.
acl:relcl	A relative clause (RC) is a clause modifying some head (typically a noun) that is understood to full-some grammatical role in the RC. The head is said to be "extracted" from the RC. Most RCs are adnominal, hence the relation acl:relcl. Adverbial RCs attach as advcl:relcl
appos	An appositional modifier of a noun is a nominal immediately following the first noun that serves to define, modify, name, or describe that noun. It includes parenthesized examples, as well as defining abbreviations in one of these structures.
amod	An adjectival modifier of a nominal is any adjective or adjectival phrase that serves to modify the meaning of the nominal.
nmod	The nmod relation is used for nominal dependents of another noun or noun phrase and functionally corresponds to an attribute, or genitive complement.
compound	There are noun compounds.
flat	The flat relation is used to combine the elements of an expression where none of the immediate components can be identified as the sole head using standard substitution tests.

Table 8: The three different metrics for accuracy

Type	Accuracy
Average Accuracy	70.6%
Accuracy-Combined	84.6%
Accuracy-UB	95.9%

Table 9: The confidence level of a question being contradictory

Confidence Level	Count of Contradictory
40%	5
60% or more	0

Table 10: F1 scores of the models for 2 and 4 fake paragraphs using GPT-4

Fake paragraph count	2	4
GPT-4 ori	87.1	87.1
GPT-4 adv	79.9 (-7.2)	77.0 (-14.1)

Table 11: Effect of self-consistency on F1 score

Prompt Style	CoT	COT+Self-consistency	COT + Instruct
Llama-2-13B ori	39.8	49.4	40.1
Llama-2-13B adv	20.4 (-19.4)	23.9 (-16.2)	23.9 (-16.2)
Llama-2-70B ori	34.4	36.0	[ILLEGIBLE]
Llama-2-70B adv	15.5	13.6	[ILLEGIBLE]